

CareerOps

Prashanth | Karthik | Ganesh

CareerOps is an AI-powered career operating system that helps job seekers manage the full job application workflow from one place. It combines master profile management, AI resume analysis, multi-agent resume tailoring, application tracking, analytics, and resume export into a single Streamlit application.

The project was created to solve a practical job-search problem: candidates often rewrite the same resume repeatedly for different job descriptions, test multiple AI tools manually, and track applications separately. CareerOps brings those steps into one structured platform where AI models and agents work like a resume writing team.

Live Documentation URL: <https://yeshiva-434245e7.mintlify.app/>

What CareerOps Does

CareerOps helps users:

- Maintain a structured master candidate profile.
- Paste or fetch a job description.
- Analyze resume-to-job fit using AI.
- Identify matching and missing job keywords.
- Run a multi-agent AI workflow for resume improvement.
- Generate tailored resume versions for each role.
- Export tailored resumes as PDF or DOCX.
- Log applications into a Kanban tracker.
- Monitor job search activity through dashboard metrics and analytics.

Core Value Proposition

Instead of using multiple AI chats manually for every job application, CareerOps centralizes the workflow:

1. Store the candidate profile once.
2. Reuse it across job descriptions.
3. Let AI agents analyze and improve the resume.
4. Track which resume version was used for each application.
5. Measure application progress over time.

This makes the job search more repeatable, measurable, and organized.

Main Components

CareerOps is built around the following major components:

Component	Purpose
Streamlit Frontend	Provides the user interface, page navigation, forms, dashboards, and visual components.
Master Profile Management	Stores candidate identity, skills, experience, education, projects, and certifications.
AI Resume Generation Engine	Analyzes job fit and generates tailored resume versions.
Multi-Agent Workflow	Uses specialized agents for extraction, research, matching, writing, and rematching.
Multi-Model Support	Supports Gemini, OpenAI, Grok, Qwen, and Ollama/local models.
PostgreSQL Database	Persists candidate profiles, applications, and activity logs.
Application Tracker	Tracks job applications through saved, applied, interviewing, offered, and rejected stages.
Dashboard and Analytics	Shows KPIs, trends, conversion rates, and resume version performance.
Export System	Converts generated resumes into PDF and DOCX formats.

System Architecture



Technology Stack

Layer	Technology
Frontend and UI	Streamlit
Backend logic	Python
Database	PostgreSQL
Database driver	psycopg2
AI orchestration	LangChain
AI providers	Gemini, OpenAI, Grok, Qwen, Ollama
Web extraction	Requests, BeautifulSoup
Data analysis	Pandas
Visualization	Plotly
PDF export	fpdf2
DOCX export	python-docx
Markdown conversion	markdown2
Secrets	Streamlit secrets, environment variables, local user_keys.json

Project Structure

```
CareerOps/
├── app.py
├── config.py
├── index.mdx
├── mint.json
├── README.md
├── requirements.txt
├── components/
│   ├── charts.py
│   ├── export_utils.py
│   ├── header.py
│   ├── kanban.py
│   ├── kpi_card.py
│   └── timeline.py
├── data/
│   ├── db_utils.py
│   └── schema.sql
├── intelligence/
│   ├── agents.py
│   ├── llm_matcher.py
│   └── resume_builder.py
├── pages/
│   ├── analytics.py
│   ├── applications.py
│   ├── dashboard.py
│   ├── generate_resume.py
│   ├── master_profile.py
│   └── settings.py
├── scripts/
│   └── init_db.py
├── .streamlit/
│   ├── config.toml
│   ├── secrets.example.toml
│   ├── secrets.toml
│   └── style.css
```

Application Pages

Dashboard

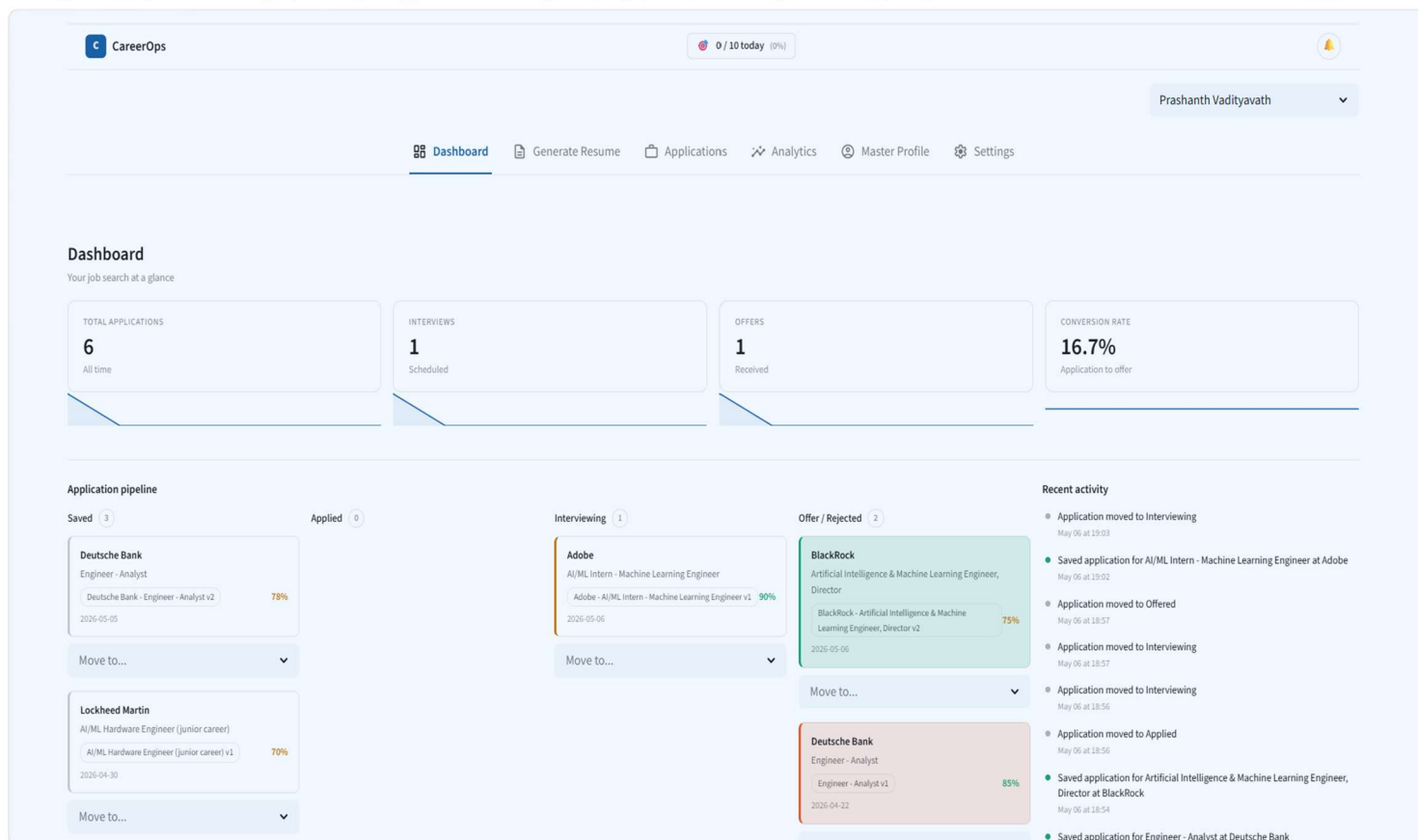
The dashboard gives a high-level view of the job search.

It includes:

- Total applications.
- Interview count.
- Offer count.
- Conversion rate.
- Weekly sparklines.
- Application pipeline preview.
- Recent activity timeline.

Relevant files:

- pages/dashboard.py
- components/kpi_card.py
- components/kanban.py
- components/timeline.py



Generate Resume

The Generate Resume page is the main AI workflow screen.

It supports:

- Job description input by paste.
- Job description fetching from URL.
- AI provider selection.
- API key configuration.
- Match score generation.
- Matching and missing keyword extraction.
- Agent-based resume upgrading.
- Resume version history.
- PDF and DOCX downloads.
- Application logging.

Relevant files:

- pages/generate_resume.py
- intelligence/resume_builder.py
- intelligence/llm_matcher.py
- intelligence/agents.py
- components/export_utils.py

CareerOps

0 / 10 today (0%)

Prashanth Vadiyavath

DashboardGenerate ResumeApplicationsAnalyticsMaster ProfileSettings

Generate Resume

Tailor your resume to a specific job description

Job URL

https://...

Fetch

or paste job description below

Paste the full job description here...

Select AI Model

Gemini

Model Name (optional)

Default: gemini-1.5-flash

> Gemini API Key Configuration

Analyze job description

Keyword Analysis

Click Analyze above to extract keywords...

Match Score

0%

Resume preview

Sync from Profile

Master Profile

PRASHANTH VADITYAVATH
prashanthvadiyavath@gmail.com | 475-317-4555
https://linkedin.com/in/prashanth-vadiyavath | https://github.com/vadiyavathprashanth

PROFESSIONAL SUMMARY

- Built enterprise applications using .NET, Angular, and SAP HANA.
- Designed ETL pipelines across finance, quality, and material systems.
- Implemented ML-based anomaly detection and predictive analytics.
- Worked with aerospace clients on data-driven solutions.

TECHNICAL SKILLS

Programming & Frameworks: Python, SQL, R, C#, JavaScript, PySpark, ASP.NET Core, Angular, React.js, Flask, FastAPI
Machine Learning & AI: TensorFlow, PyTorch, Scikit-Learn, XGBoost, NLP, Deep Learning, ARIMA, GARCH

Company name

Role title

Print View

Save and log application

Export to PDF or Word document

Applications

The Applications page provides a Kanban board for tracking job applications.

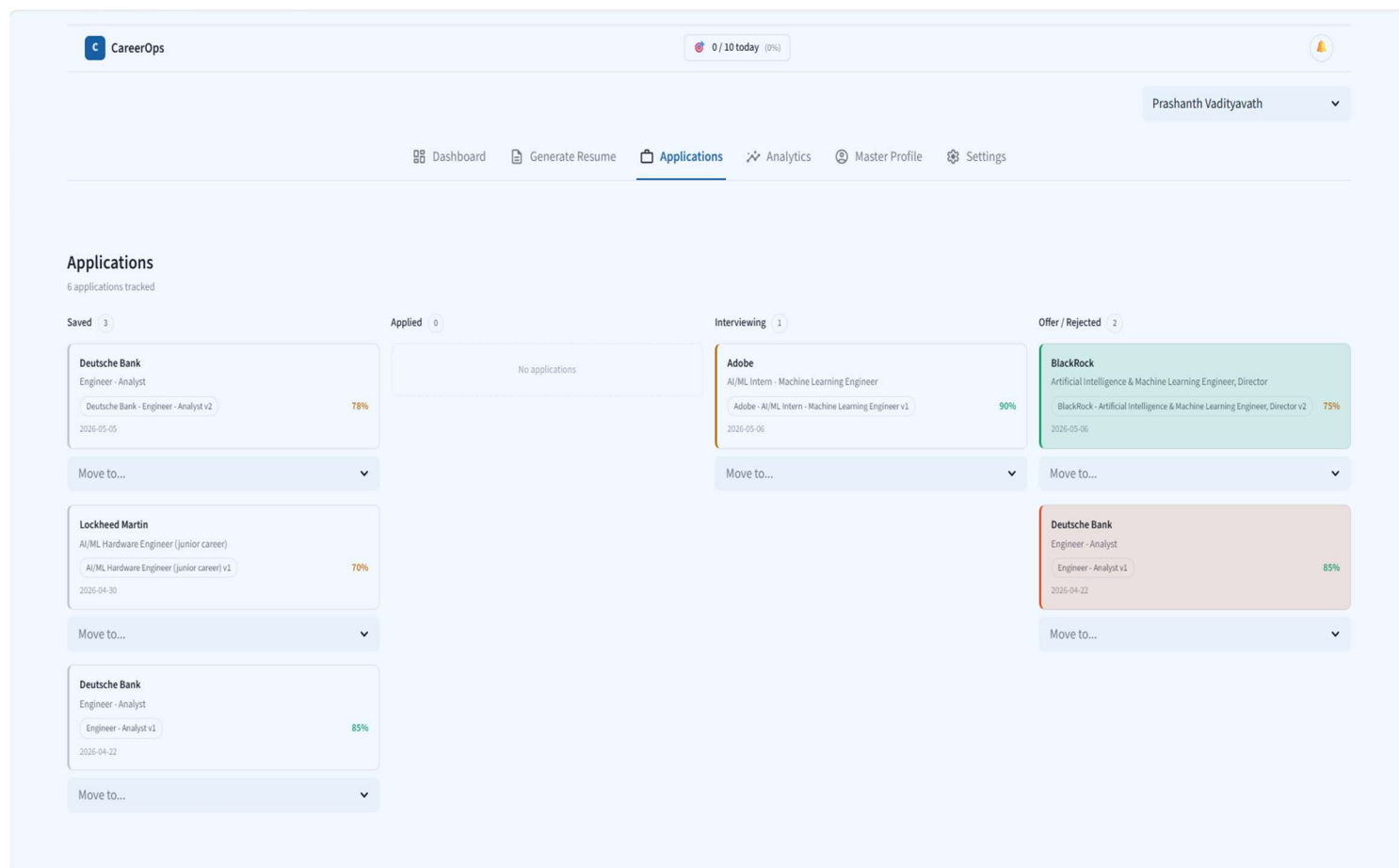
Supported stages:

- Saved
- Applied
- Interviewing
- Offered
- Rejected

Users can move applications between stages. Each movement updates the database and records activity in the timeline.

Relevant files:

- pages/applications.py
- components/kanban.py
- data/db_utils.py



Master Profile

The Master Profile page manages the candidate profile that powers resume generation.

It supports CRUD operations for:

- Candidate details.
- Professional summary.
- Technical skills.
- Work experience.
- Education.
- Projects.
- Certifications.

Relevant files:

- pages/master_profile.py
- data/db_utils.py
- data/schema.sql

CareerOps

0 / 10 today (10%)

Prashanth Vadiyavath

DashboardGenerate ResumeApplicationsAnalyticsMaster ProfileSettings

Master Profile

Manage candidate profiles, skills, experience, and credentials

Create new candidate

Candidate details

Basic information

Full name

Email

Phone

Links

LinkedIn URL

GitHub URL

Portfolio URL

Objective and summary

Career objective

Professional summary -- one bullet per line

Save candidate

Select or create a candidate above to manage their profile sections.

CareerOps

0 / 10 today (2%)

Prashanth Vadiyavath

DashboardGenerate ResumeApplicationsAnalyticsMaster ProfileSettings

Master Profile

Manage candidate profiles, skills, experience, and credentials

Prashanth Vadiyavath

Candidate details

Basic information

Full name

Prashanth Vadiyavath

Email

prashanthvadiyavath@gmail.com

Phone

475-317-4555

Links

LinkedIn URL

https://linkedin.com/in/prashanth-vadiyavath

GitHub URL

https://github.com/vadiyavathprashanth

Portfolio URL

Objective and summary

Career objective

Software developer and data engineer focused on building scalable data pipelines, ML systems, and enterprise applications.

Professional summary — one bullet per line

Built enterprise applications using .NET, Angular, and SAP HANA.
Designed ETL pipelines across finance, quality, and material systems.
Implemented ML-based anomaly detection and predictive analytics.

Save candidate

CareerOps

0 / 10 today (2%)

Prashanth Vadiyavath

DashboardGenerate ResumeApplicationsAnalyticsMaster ProfileSettings

Master Profile

Manage candidate profiles, skills, experience, and credentials

Prashanth Vadiyavath

Candidate details

Managing profile for Prashanth Vadiyavath

Technical skills

Work experience

Education

Projects

Certifications

Settings

The Settings page contains configuration and diagnostics.

It supports:

- Daily application goal management.
- Database connection test.
- Streamlit theme guidance.
- Account placeholder fields for future authentication.

Relevant files:

- `pages/settings.py`
- `config.py`
- `data/db_utils.py`

CareerOps 0 / 10 today (0%) Prashanth Vadityavath

Dashboard Generate Resume Applications Analytics Master Profile **Settings**

Settings

Preferences and system diagnostics

Account

Placeholder -- authentication coming in Phase 4.

Email: john.doe@example.com Display name: John Doe

Save account

Daily application goal

Sets the target shown in the header progress bar.

Target applications per day: 10

Save goal

Appearance

Theme is controlled by Streamlit itself.

To switch between light and dark mode, open the Streamlit menu (top right corner) and go to Settings > Theme.

Database diagnostics

Verify your PostgreSQL connection and schema.

Test database connection

AI Workflow

CareerOps uses two AI layers:

6. A direct LLM matcher for structured job fit analysis.
7. A LangChain multi-agent workflow for deeper resume improvement.

LLM Matcher

The matcher compares the candidate resume text with a job description.

It returns:

- Match score from 0 to 100.
- Matching keywords.
- Missing keywords.
- Requirement checklist.
- Short recommendation.

Relevant file:

- `intelligence/llm_matcher.py`

Multi-Agent Workflow

CareerOps treats AI as a small resume writing team. Each agent has a focused responsibility.

Agent	Responsibility
Extractor Agent	Extracts company name and job title from the job description.
Researcher Agent	Looks for company, culture, and role-related context.
Matcher Agent	Identifies resume gaps compared with the job description.
Writer Agent	Rewrites the resume while preserving factual accuracy.
Rematcher Agent	Scores the updated resume and shows improvement.

Relevant file:

- `intelligence/agents.py`

Supported AI Providers

CareerOps supports both cloud and local model providers.

Provider	Default Model In Code	Notes
Gemini	gemini-1.5-flash	Good for fast job matching and general reasoning.
OpenAI	gpt-4o	Strong general-purpose model support.
Grok	llama3-groq-70b-8192-tool-use-preview	Uses Groq-compatible client flow.
Qwen	qwen-plus	Uses OpenAI-compatible DashScope endpoint.
Ollama	llama3	Supports local model execution through OpenAI-compatible API.

API credentials can be loaded from:

- Streamlit session state.
- user_keys.json.
- .streamlit/secrets.toml.
- Environment variables.

Resume Versioning and Export

CareerOps maintains resume versions in Streamlit session state per active candidate.

The base version is:

- Master Profile

AI-generated versions use company and role context when available, for example:

- Google - Data Analyst v1
- Google - Data Analyst v2

Downloads use meaningful filenames:

- CareerOps_Google_Data_Analyst_v1.pdf
- CareerOps_Google_Data_Analyst_v1.docx

Export formats:

- PDF through fpdf2.
- DOCX through python-docx.

Relevant file:

- components/export_utils.py

Database Design

CareerOps uses PostgreSQL as the persistent data store.

The schema is normalized around a candidate profile and job application tracking model.

Tables

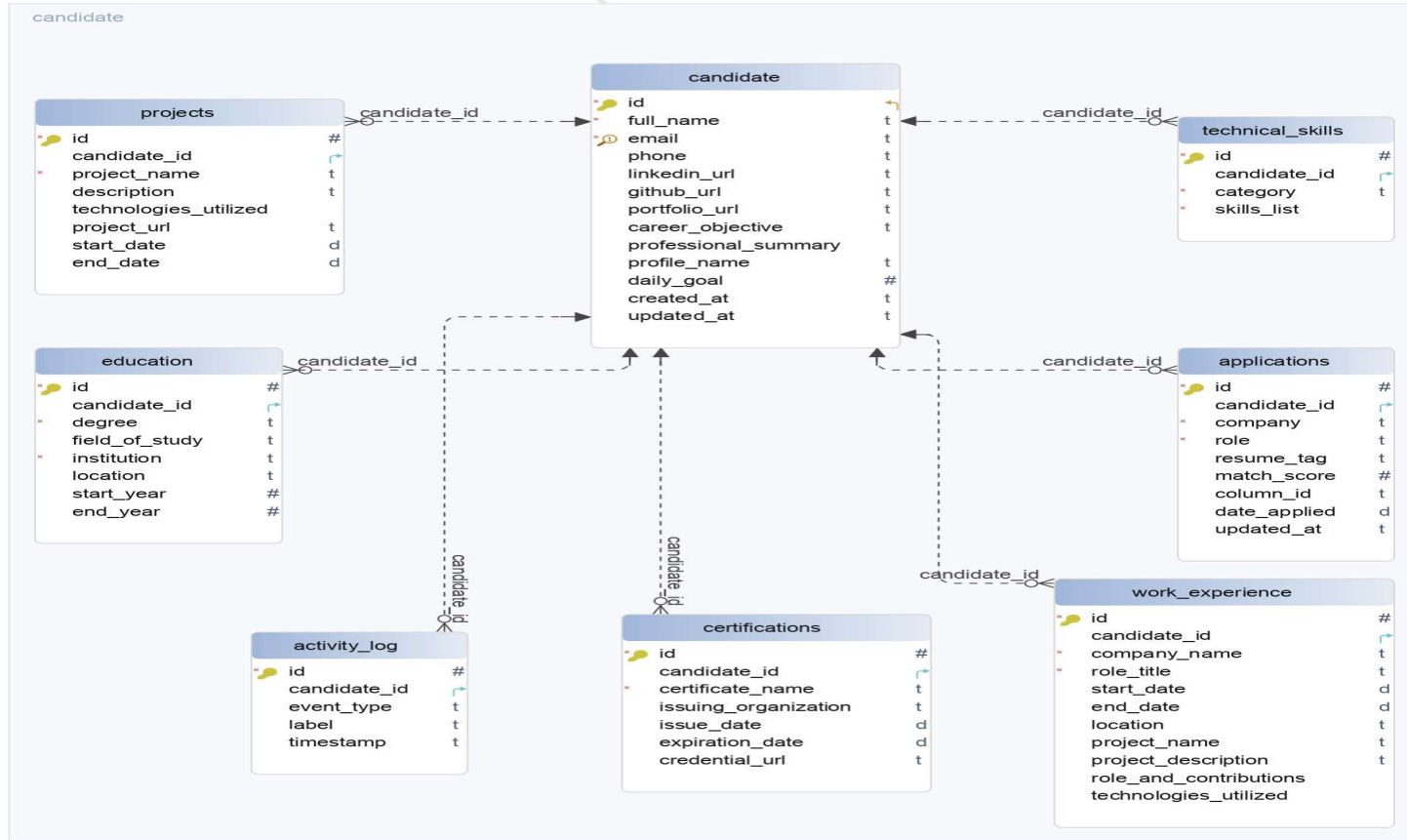
Table	Purpose
candidate	Stores core identity, contact links, career objective, professional summary, and daily goal.
technical_skills	Stores categorized skill lists for each candidate.
work_experience	Stores role history, company names, dates, contributions, and technologies.
education	Stores education records.
projects	Stores portfolio projects and technologies.
certifications	Stores certificate and credential records.
applications	Stores company, role, resume version, match score, pipeline status, and application date.
activity_log	Stores timeline events such as application logging and status movement.

Relevant files:

- data/schema.sql
- data/db_utils.py

Main Diagram

12-05-2026 DbSchema.com © DATE Wise Code



Main Diagram

Table activity_log		
Idx	Name	Data Type
* Pk	id	integer DEFAULT nextval('activity_log_id_seq'::regclass)
	candidate_id	integer
	event_type	varchar(50)
	label	text
	timestamp	timestamp DEFAULT CURRENT_TIMESTAMP
Indexes		
Type	Name	On
Pk	activity_log_pkey	id
Foreign Keys		
Type	Name	On
	activity_log_candidate_id_fkey (candidate_id) ref candidate (id)	

Table applications		
Idx	Name	Data Type
* Pk	id	integer DEFAULT nextval('applications_id_seq'::regclass)
	candidate_id	integer
*	company	varchar(255)
*	role	varchar(255)
	resume_tag	varchar(255)
	match_score	integer
	column_id	varchar DEFAULT 'saved'::character varying
	date_applied	date DEFAULT CURRENT_DATE
	updated_at	timestamp DEFAULT CURRENT_TIMESTAMP
Indexes		
Type	Name	On
Pk	applications_pkey	id
Foreign Keys		
Type	Name	On
	applications_candidate_id_fkey (candidate_id) ref candidate (id)	

Table candidate		
Idx	Name	Data Type
* Pk	id	integer DEFAULT nextval('candidate_id_seq'::regclass)
*	full_name	varchar(255)
* Unq	email	varchar(255)
	phone	varchar(50)
	linkedin_url	varchar(255)
	github_url	varchar(255)
	portfolio_url	varchar(255)
	career_objective	text
	professional_summary	jsonb
	profile_name	varchar DEFAULT 'Default Profile'::character varying
	daily_goal	integer DEFAULT 5
	created_at	timestamp DEFAULT CURRENT_TIMESTAMP
	updated_at	timestamp DEFAULT CURRENT_TIMESTAMP

Table candidate		
Indexes		
Type	Name	On
Pk	candidate_pkey	id
Unq	candidate_email_key	email

Table certifications		
Idx	Name	Data Type
* Pk	id	integer DEFAULT nextval('certifications_id_seq'::regclass)
	candidate_id	integer
*	certificate_name	varchar(255)
	issuing_organization	varchar(255)
	issue_date	date
	expiration_date	date
	credential_url	varchar(255)
Indexes		
Type	Name	On
Pk	certifications_pkey	id
Foreign Keys		
Type	Name	On
	certifications_candidate_id_fkey (candidate_id) ref candidate (id)	

Table education		
Idx	Name	Data Type
* Pk	id	integer DEFAULT nextval('education_id_seq'::regclass)
	candidate_id	integer
*	degree	varchar(255)
	field_of_study	varchar(255)
*	institution	varchar(255)
	location	varchar(255)
	start_year	integer
	end_year	integer
Indexes		
Type	Name	On
Pk	education_pkey	id
Foreign Keys		
Type	Name	On
	education_candidate_id_fkey (candidate_id) ref candidate (id)	

Table projects		
Idx	Name	Data Type
* Pk	id	integer DEFAULT nextval('projects_id_seq'::regclass)
	candidate_id	integer
*	project_name	varchar(255)
	description	text
	technologies_utilized	jsonb
	project_url	varchar(255)
	start_date	date
	end_date	date

Table projects

Indexes		
Type	Name	On
Pk	projects_pkey	id
Foreign Keys		
Type	Name	On
	projects_candidate_id_fkey (candidate_id) ref candidate (id)	

Table technical_skills

Idx	Name	Data Type
* Pk	id	integer DEFAULT nextval('technical_skills_id_seq'::regclass)
	candidate_id	integer
*	category	varchar(100)
*	skills_list	jsonb
Indexes		
Type	Name	On
Pk	technical_skills_pkey	id
Foreign Keys		
Type	Name	On
	technical_skills_candidate_id_fkey (candidate_id) ref candidate (id)	

Table work_experience

Idx	Name	Data Type
* Pk	id	integer DEFAULT nextval('work_experience_id_seq'::regclass)
	candidate_id	integer
*	company_name	varchar(255)
*	role_title	varchar(255)
	start_date	date
	end_date	date
	location	varchar(255)
	project_name	text
	project_description	text
	role_and_contributions	jsonb
	technologies_utilized	jsonb
Indexes		
Type	Name	On
Pk	work_experience_pkey	id
Foreign Keys		
Type	Name	On
	work_experience_candidate_id_fkey (candidate_id) ref candidate (id)	

Database Utility Layer

data/db_utils.py centralizes database access.

It provides:

- Cached PostgreSQL connection creation.
- Connection health checks and reconnect behavior.
- Parameterized query execution.
- Transaction commit and rollback handling.
- Candidate profile fetching.
- Application CRUD helpers.
- Activity logging.
- Tracking table setup.

Setup and Installation

Prerequisites

Install the following before running CareerOps:

- Python 3.11 or newer.
- PostgreSQL.
- API key for at least one supported cloud model, unless using Ollama locally.

1. Clone the Repository

Code block:

```
git clone <your-repo-url>
cd CareerOps
```

2. Create a Virtual Environment

Windows:

Code block:

```
python -m venv .venv
.venv\Scripts\activate
```

macOS or Linux:

Code block:

```
python3 -m venv .venv
source .venv/bin/activate
```

3. Install Dependencies

Code block:

```
pip install -r requirements.txt
```

4. Configure PostgreSQL Secrets

Copy the example secrets file:

Code block:

```
cp .streamlit/secrets.example.toml .streamlit/secrets.toml
```

On Windows PowerShell:

Code block:

```
Copy-Item .streamlit\secrets.example.toml .streamlit\secrets.toml
```

Edit .streamlit/secrets.toml:

Code block: toml

```
[postgres]
host = "localhost"
port = 5432
dbname = "careerops"
user = "postgres"
password = "your_password"
```

5. Initialize the Database

Run:

Code block:

```
python scripts/init_db.py
```

Optional commands:

Code block:

```
python scripts/init_db.py --db-only
python scripts/init_db.py --schema-only
```

6. Run the Application

Code block:

```
streamlit run app.py
```

By default, Streamlit opens the app at:

Code block:

```
http://localhost:8050
```

Configuration

API Keys

API keys can be configured in multiple ways:

8. Enter them through the application UI.
9. Save them in local user_keys.json.
10. Add them to .streamlit/secrets.toml.
11. Export them as environment variables.

Expected key patterns:

Code block: text

```
GEMINI_API_KEY  
OPENAI_API_KEY  
GROK_API_KEY  
QWEN_API_KEY  
OLLAMA_BASE_URL
```

For Ollama, the default base URL is:

Code block: text

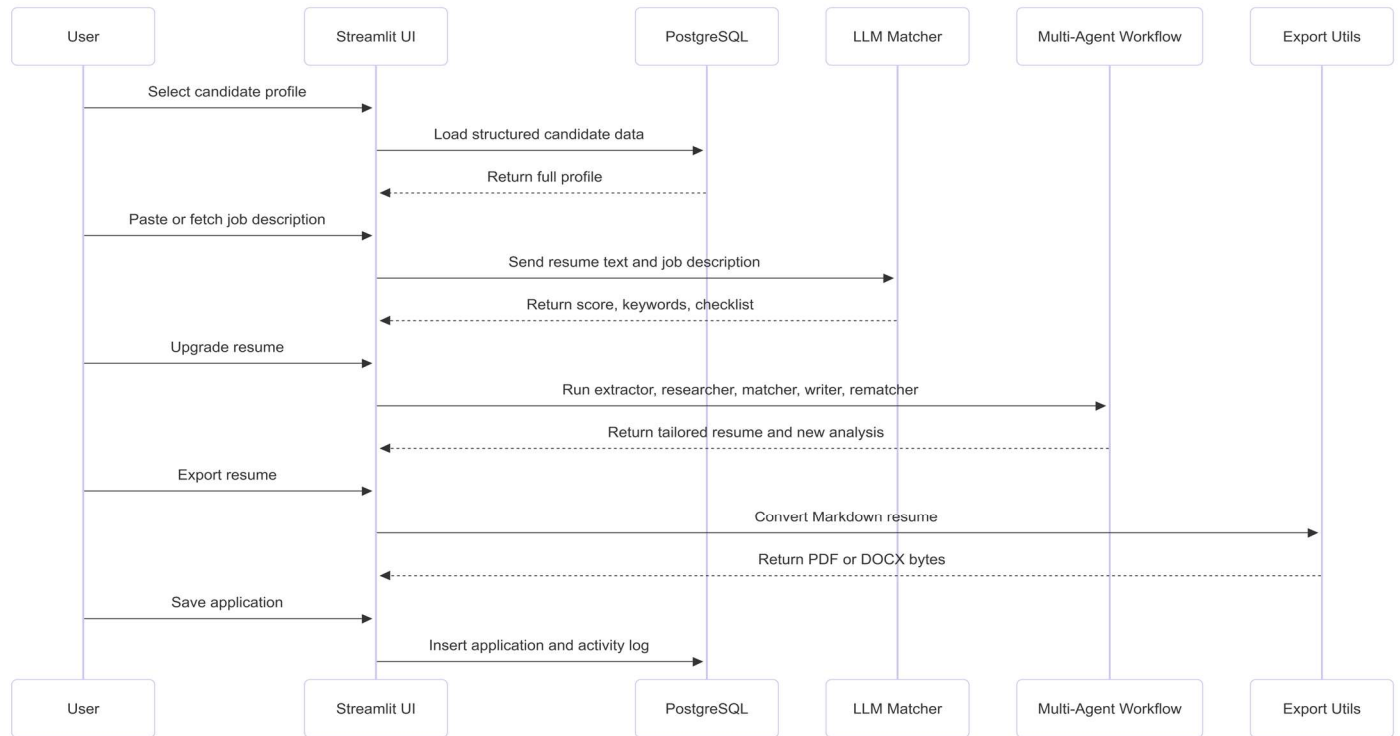
```
http://localhost:8050
```

User Workflow

Generate and Track a Tailored Resume

12. Open CareerOps.
13. Create or select a candidate profile.
14. Add profile details in Master Profile.
15. Go to Generate Resume.
16. Paste a job description or fetch it from a URL.
17. Select an AI model provider.
18. Run the job match analysis.
19. Review matching and missing keywords.
20. Run the resume upgrade workflow.
21. Review the tailored resume version.
22. Export the resume as PDF or DOCX.
23. Save and log the application.
24. Track the application in the Kanban board.
25. Review dashboard and analytics over time.

Data Flow



Security and Secrets

CareerOps handles sensitive data such as:

- Candidate profile information.
- Resume content.
- Job descriptions.
- API keys.
- Database credentials.

Security guidelines:

- Do not commit `.streamlit/secrets.toml`.
- Do not commit `user_keys.json`.
- Do not commit `.env`.
- Use environment variables or secure secrets management for deployment.
- Be cautious with AI tracing tools, because resume and job description content may contain personal data.
- Use database credentials with the minimum required privileges in production.

The repository `.gitignore` excludes local secret files.

Current Limitations

CareerOps is functional, but several areas can be improved:

- Streamlit is convenient for rapid development, but complex dynamic UI behavior is harder to maintain.
- Resume export formatting can be improved with stronger document templates.
- Session-state version history is useful during a session, but long-term resume version storage should move into the database.
- AI calls can consume many tokens when resumes and job descriptions are long.
- RAG-based memory is not yet implemented.
- Authentication is currently a placeholder.

Future Direction

Planned architectural improvements include:

- Move complex frontend interactions to React and JavaScript.
- Keep Python services for AI workflows, database operations, and document generation.
- Persist resume versions and AI analysis history in the database.
- Add RAG architecture for long-term resume memory and context retrieval.
- Improve PDF and DOCX templates for professional resume alignment.
- Add authentication and user-specific data isolation.
- Add browser extension support for sending job descriptions directly from job boards.
- Add email integration to update application stages from interview invites.

RAG Architecture Plan

RAG can improve CareerOps by storing and retrieving relevant career context instead of sending all historical information to the model every time.

Future RAG storage could include:

- Master profile snapshots.
- Generated resume versions.
- Job descriptions.
- Match analyses.
- Interview outcomes.
- Application status history.
- User feedback on generated resumes.

Expected benefits:

- Lower token usage.
- Better long-term personalization.
- More consistent AI output.
- Smarter recommendations based on previous applications.
- Ability to answer questions about past job search performance.

Troubleshooting

Database connection fails

Check:

- PostgreSQL is running.
- `.streamlit/secrets.toml` exists.
- The `[postgres]` section has host, port, dbname, user, and password.
- The database was initialized with `python scripts/init_db.py`.

AI analysis fails

Check:

- The selected provider API key is configured.
- The model name is valid.
- The provider has not hit rate limits.
- The prompt input is not too large for the model.
- Ollama is running locally if using local models.

Job URL fetch fails

Some job boards block automated requests. If fetching fails, paste the job description manually into the text area.

Export formatting looks imperfect

The current export path converts Markdown to PDF or DOCX. For best final formatting, future versions should use dedicated resume templates or a more controlled rendering pipeline.

Summary

CareerOps is a complete AI-assisted job search platform. It combines structured profile management, multi-model AI support, multi-agent resume tailoring, PostgreSQL persistence, application tracking, analytics, and document export.

The project demonstrates practical AI application development: prompt design, model integration, token and cost awareness, database architecture, frontend tradeoffs, and a clear path toward future RAG-powered personalization.